Chapter 2 — AI Engineering Fundamentals

2.1 From Deterministic to Probabilistic Software

Conceptual Deep Dive

Traditional software maps input to output through code you wrote; the same input gives the same output and tests assert equality. An LLM samples tokens from a probability distribution `P(next token | context)`. Even at temperature 0, batching, floating-point non-associativity and provider-side changes make outputs **non-deterministic** in practice. Consequences:

- Tests become **statistical**: run N samples, assert a pass rate.
- Correctness is a **spectrum** (faithful, relevant, safe, well-formatted), so you build graded evals, not boolean asserts.
- Behaviour **drifts** when the provider updates a model alias.
- This non-determinism is inherent, not a bug to be eliminated.
- Failure modes are semantic (confident wrong answers), not exceptions.

Sampling controls: **temperature** rescales logits (`softmax(z/T)`); low T sharpens the distribution, high T flattens it. **top-p** keeps the smallest set of tokens with cumulative probability ≥ p. **top-k** keeps the k most likely. **Seed** helps reproducibility where supported but is not a guarantee.

Cross-Connection Scenarios

Because outputs vary, Chapter 7 (validation/retries) and Chapter 10 (statistical evals) exist. Chapter 15 turns eval pass-rates into CI gates.

Production Anti-Patterns

- `assert response == "expected"` tests that flake.
- Pinning to a floating alias (`latest`) and being surprised by regressions.
- Parsing free text with regex instead of constraining output.

Minimal Working Code

```
import math
import random

def softmax(logits, temperature=1.0):
    scaled = [z / max(temperature, 1e-6) for z in logits]
    m = max(scaled)
    exps = [math.exp(z - m) for z in scaled]
    s = sum(exps)
    return [e / s for e in exps]

def sample(probs, top_p=1.0, rng=random):
    order = sorted(range(len(probs)), key=lambda i: -probs[i])
    kept, cum = [], 0.0
    for i in order:
        kept.append(i)
        cum += probs[i]
        if cum >= top_p:
            break
```

```
    weights = [probs[i] for i in kept]
    return rng.choices(kept, weights=weights)[0]

logits = [2.0, 1.0, 0.1]
print(softmax(logits, 0.2), softmax(logits, 2.0))
print(sample(softmax(logits), top_p=0.9))
```

2.2 The Modern AI Stack

Conceptual Deep Dive

Layers, bottom to top: **hardware** (GPU/TPU) → **runtime/serving** (vLLM, TGI) → **models** (closed API or open weights) → **orchestration** (prompts, tools, agents, RAG) → **data** (vector stores, feature stores, caches) → **evaluation & observability** → **application/UX**. Decisions at each layer trade cost, latency, quality, privacy and control.

Key budgets: **context window** (max tokens in+out), **latency** (time-to-first-token and tokens/sec), **cost** (price per million input/output tokens; output usually costs more), **throughput** (requests/sec under GPU memory constraints).

Cross-Connection Scenarios

Model choice (closed vs open) links to fine-tuning (12), serving (13) and security/data residency (14). Context window limits drive chunking and retrieval (9).

Production Anti-Patterns

- Using the largest model for every call; route easy tasks to small models.
- Stuffing the entire corpus into context ("long context replaces RAG") without measuring cost and lost-in-the-middle degradation.
- No fallback provider.

Minimal Working Code

A model router keyed on task difficulty:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ModelSpec:
    name: str
    cost_per_mtok_in: float
    cost_per_mtok_out: float
    max_context: int

SMALL = ModelSpec("small-model", 0.1, 0.4, 128_000)
LARGE = ModelSpec("large-model", 3.0, 15.0, 200_000)

def route(task: str, prompt_tokens: int) -> ModelSpec:
    hard = any(k in task for k in ("reason", "plan", "code", "legal&qu
ot;))
    if prompt_tokens > SMALL.max_context or hard:
        return LARGE
    return SMALL

def estimate_cost(m: ModelSpec, tin: int, tout: int) -> float:
    return tin / 1e6 * m.cost_per_mtok_in + tout / 1e6 * m.cost_per_mtok_out

m = route("summarize", 2_000)
print(m.name, estimate_cost(m, 2_000, 300))
```

2.3 Developer Mental Models

Conceptual Deep Dive

1. **The model is a function of its context.** It has no memory beyond what you send. "Memory" is something you build (Chapter 11).
2. **Prompts are programs; evals are tests.** Version both.
3. **Capability ladder:** prompt → few-shot → RAG → tools → fine-tune → train. Escalate only on eval evidence.
4. **Cost-latency-quality triangle:** you can optimise two; caching, smaller models and batching shift the frontier.
5. **Data flywheel:** logs → failures → eval cases → improvements.

Cross-Connection Scenarios

The flywheel connects observability (14) → evaluation (10) → CI (15). The capability ladder maps to chapters 6 → 9 → 11 → 12.

Production Anti-Patterns

- Jumping to fine-tuning to fix a retrieval problem.
- Believing the model "knows" your private data.
- Shipping without a way to capture failing examples.

Minimal Working Code

```
import json
import time
from pathlib import Path

LOG = Path("interactions.jsonl")

def log_interaction(prompt: str, output: str, meta: dict | None = None) -> None:
    rec = {"ts": time.time(), "prompt": prompt, "output": output, **(meta o
r {})}
    with LOG.open("a", encoding="utf-8") as f:
        f.write(json.dumps(rec) + "\n")

def promote_to_eval(rec_index: int, expected: str, out: Path = Path("eval_cases.jsonl")) ->
None:
    recs = [json.loads(l) for l in LOG.read_text(encoding="utf-8").splitlines()]
    case = {"input": recs[rec_index]["prompt"], "expected": expected}
    with out.open("a", encoding="utf-8") as f:
        f.write(json.dumps(case) + "\n")
```

2.4 Token Economics and Latency

Conceptual Deep Dive

Tokens, not characters, are billed. English averages ~0.75 words/token; code and non-Latin scripts are costlier. Latency = queueing + prefill (processes the prompt in parallel, compute-bound) + decode (one token at a time, memory-bandwidth-bound). TTFT is dominated by prefill; inter-token latency by decode. **Prompt caching** reuses KV state for repeated prefixes, cutting cost and TTFT.

Cross-Connection Scenarios

Streaming (4) improves perceived latency; KV cache (5) and continuous batching (13) explain provider-side behaviour.

Production Anti-Patterns

- Putting volatile data at the start of the prompt, destroying prefix caching.
- Unbounded max_tokens.
- Ignoring output tokens in cost models.

Minimal Working Code

```
def budget(requests_per_day: int, tin: int, tout: int, p_in: float, p_
out: float) -> float:
    per_req = tin / 1e6 * p_in + tout / 1e6 * p_out
    return per_req * requests_per_day * 30

print(f"${budget(50_000, 1500, 400, 0.5, 2.0):,.2f}/month")
```

2.5 Failure Taxonomy of LLM Systems

Conceptual Deep Dive

Classify failures to fix the right layer: **knowledge** (stale/missing facts), **reasoning** (multi-step slips, arithmetic), **instruction** (ignored constraints, format drift), **retrieval** (wrong/missing context), **tooling** (bad arguments, wrong tool), **safety** (injection, harmful output), **systems** (timeouts, rate limits, truncation) and **UX** (ambiguous queries, missing clarification). Edge cases: silent truncation when input exceeds the context window; finish_reason == "length" producing cut-off JSON; refusals that look like answers; sycophancy (agreeing with a wrong user premise); positional bias; and "format collapse" in long outputs. Tagging each failed eval case with one category turns a pile of anecdotes into a prioritised backlog.

Cross-Connection Scenarios

Retrieval failures route to Chapters 8–9; instruction failures to 6–7; safety to 14; systems to 4 and 13.

Production Anti-Patterns

- Responding to every failure by editing the prompt.
- Ignoring finish_reason and parsing truncated output.
- No tagging of failures, so no way to prioritise.

Minimal Working Code

```
from collections import Counter

CATEGORIES = {"knowledge", "reasoning", "instruction", "retrieval"
;, "tooling", "safety", "systems", "ux"}

def triage(failures: list[dict]) -> list[tuple[str, int]]:
    bad = [f for f in failures if f["category"] not in CATEGORIES]
    if bad:
        raise ValueError(f"unknown categories: {bad[:2]}")
    return Counter(f["category"] for f in failures).most_common()

print(triage([{"category": "retrieval"}, {"category": "retrieval"
}, {"category": "instruction"}]))
```

```
def check_finish(resp: dict) -> str:
    """Never parse a response that was cut off."""
    reason = resp.get("finish_reason")
    if reason == "length":
        raise ValueError("output truncated: raise max_tokens or shorten the task")
    if reason == "content_filter":
        raise PermissionError("provider filtered the response")
    return resp["text"]

print(check_finish({"finish_reason": "stop", "text": "ok"}))
```

2.6 Build vs Buy and Vendor Strategy

Conceptual Deep Dive

Decide per layer: use a **hosted API** for fastest time to value and frontier quality; **managed open-model hosting** for cost with less ops; **self-hosting** for data control, latency, or volume. Mitigate lock-in with a thin provider-agnostic interface, prompts that avoid vendor quirks, portable eval suites, and a tested secondary provider. Consider data-processing terms (retention, training on your data), regional availability, rate-limit tiers, deprecation policies, and SLAs. Edge cases: a provider retiring a model with 3–6 months' notice, silent quality changes behind an alias, and regional outages.

Cross-Connection Scenarios

Gateways (13.6), fallbacks (4.4), fine-tuned replacements (12, 17.5), legal/security review (14.5).

Production Anti-Patterns

- Spreading vendor SDK types through the whole codebase.
- Having a "backup provider" that was never load-tested.
- Ignoring deprecation announcements until the cutoff date.

Minimal Working Code

```
from typing import Protocol

class ChatModel(Protocol):
    def chat(self, messages: list[dict], **kw) -> str: ...

class ProviderA:
    def chat(self, messages, **kw):
        return "from A"

class ProviderB:
    def chat(self, messages, **kw):
        return "from B"

def chat_with_fallback(models: list[ChatModel], messages):
    last = None
    for m in models:
        try:
            return m.chat(messages)
        except Exception as e:
            last = e
    raise RuntimeError("all providers failed") from last

print(chat_with_fallback([ProviderA(), ProviderB()], [{"role": "user", "content&
quot;: "hi"}]))
```

```
from datetime import date

DEPRECATIONS = {"old-model": date(2026, 12, 31)}

def deprecation_warnings(used: list[str], today: date, warn_days: int = 120):
    return [f"{m} retires {d} ({(d - today).days}d)" for m, d in DEPRECATIONS.items()
            if m in used and (d - today).days <= warn_days]

print(deprecation_warnings(["old-model"], date(2026, 10, 22)))
```